Postgres and Python for the visualization of Spatial Data

Advanced Topics in Databases

Psycopg – PostgreSQL database adapter for Python

- Psycopg is the most popular PostgreSQL database adapter for the Python programming language.
- Its main features are the complete implementation of the Python DB API 2.0 specification and the thread safety (several threads can share the same connection).
- It was designed for heavily multi-threaded applications that create and destroy lots of cursors and make a large number of concurrent INSERTs or UPDATEs.
- Psycopg 2 is mostly implemented in C as a libpq wrapper, resulting in being both efficient and secure.
- Many Python types are supported out-of-the-box and adapted to matching PostgreSQL data types;

The psycopg2 module content

- The module interface respects the standard defined in the DB API 2.0.

  psycopg2.connect(dsn=None, connection_factory=None, cursor_factory=None, async=False, **kwargs)

- Create a new database session and return a new connection object.
- The connection parameters can be specified as a libpq connection string using the dsn parameter:

```
conn = psycopg2.connect("dbname=test user=postgres password=secret")
```

- or using a set of keyword arguments:

```
conn = psycopg2.connect(dbname="test", user="postgres", password="secret")
```

connect()

- The basic connection parameters are:
  - dbname – the database name (database is a deprecated alias)
  - user – user name used to authenticate
  - password – password used to authenticate
  - host – database host address (defaults to UNIX socket if not provided)
  - port – connection port number (defaults to 5432 if not provided)

The connection class

- Handles the connection to a PostgreSQL database instance. It encapsulates a database session.
- Connections are created using the factory function connect().
- Connections are thread safe and can be shared among many threads. See Thread and process safety for details.

connection.cursor() method

cursor(name=None, cursor_factory=None, scrollable=None, withhold=False)

- Return a new cursor object using the connection.
- If name is specified, the returned cursor will be a server side cursor (also known as named cursor). Otherwise it will be a regular client side cursor.

connection.commit(), connection.rollback(), connection.close()

commit()

- Commit any pending transaction to the database.
- By default, Psycopg opens a transaction before executing the first command: if commit() is not called, the effect of any data manipulation will be lost.
- The connection can be also set in “autocommit” mode: no transaction is automatically open, commands have immediate effect.

rollback()

- Roll back to the start of any pending transaction. Closing a connection without committing the changes first will cause an implicit rollback to be performed.

close()

- Close the connection now. The connection will be unusable from this point forward

The cursor class

- Allows Python code to execute PostgreSQL command in a database session. Cursors are created by the connection.cursor() method: they are bound to the connection for the entire lifetime and all the commands are executed in the context of the database session wrapped by the connection.

- Cursors created from the same connection are not isolated, i.e., any changes done to the database by a cursor are immediately visible by the other cursors. Cursors created from different connections can or can not be isolated, depending on the connections' isolation level.

close() method

- Close the cursor now. The cursor will be unusable from this point forward; an InterfaceError will be raised if any operation is attempted with the cursor.

Commands execution methods

execute(query, vars=None)

- Execute a database operation (query or command).
- Parameters may be provided as sequence or mapping and will be bound to variables in the operation. Variables are specified either with positional (%s) or named (%(name)s) placeholders.
- The method returns None. If a query was executed, the returned values can be retrieved using fetch*() methods.

executemany(query, vars_list)

- Execute a database operation (query or command) against all parameter tuples or mappings found in the sequence vars_list.
- The function is mostly useful for commands that update the database: any result set returned by the query is discarded.
- Parameters are bounded to the query using the same rules described in the execute() method.

```
>>> nums = ((1,), (5,), (10,))
>>> cur.executemany("INSERT INTO test (num) VALUES (%s)", nums)

>>> tuples = ((123, "foo"), (42, "bar"), (23, "baz"))
>>> cur.executemany("INSERT INTO test (num, data) VALUES (%s, %s)", tuples)
```

Results retrieval methods

- The following methods are used to read data from the database after an execute() call:

fetchone()

- Fetch the next row of a query result set, returning a single tuple, or None when no more data is available:

```
>>> cur.execute("SELECT * FROM test WHERE id = %s", (3,))
>>> cur.fetchone()
(3, 42, 'bar')
```

- A programming error is raised if the previous call to execute*() did not produce any result set or no call was issued yet.

Results retrieval methods

fetchall()

- Fetch all (remaining) rows of a query result, returning them as a list of tuples. An empty list is returned if there is no more record to fetch.

```
>>> cur.execute("SELECT * FROM test;")
>>> cur.fetchall()
[(1, 100, "abc'def"), (2, None, 'dada'), (3, 42, 'bar')]
```

- A programming error is raised if the previous call to execute*() did not produce any result set or no call was issued yet.

Adaptation of Python values to SQL types

- Many standard Python types are adapted into SQL and returned as Python objects when a query is executed.
- The following table shows the default mapping between Python and PostgreSQL types:

Python	PostgreSQL
None	NULL
bool	bool
float	real double
int long	smallint integer bigint
Decimal	numeric
str unicode	varchar text
buffer memoryview bytearray bytes Buffer protocol	bytea
date	date
time	time timetz
datetime	timestamp timestamptz
timedelta	interval
list	ARRAY
tuple namedtuple	Composite types IN syntax
dict	hstore
Psycopg's Range	range
Anything™	json
UUID	uuid
ipaddress objects	inet cidr

Further study

https://www.psycopg.org/docs/index.html

Visualization of spatial data with Python and modules Matplotlib, Shapely and Contextily

- Matplotlib is a powerful and very popular data visualization library in Python.
- Documentation and examples available at:

  https://matplotlib.org/stable/index.html

Visualization of spatial data with Python and modules Matplotlib, Shapely and Contextily

- Matplotlib is a powerful and very popular data visualization library in Python.
- Shapely is a Python library that allows creating and analyzing planar geometric objects.
- Contextily is a Python library that adds map tiles as background for our plots.

What is shapely?

- A Python library for:
  - Creating and analyzing **planar geometric objects**
  - Based on the **GEOS** library (same engine as PostGIS)
- Supports:
  - Point, LineString, Polygon, Multi*, GeometryCollection
  - Operations: .area, .buffer(), .intersects(), etc.

What is matplotlib?

- A versatile plotting library for Python
- Can visualize:
  - Coordinates from Shapely geometries
  - Custom 2D spatial data
- Commonly used alongside:
  - shapely, geopandas, contextily, etc.

Why Use Shapely + Matplotlib?

- Lightweight alternative to full GIS platforms
- Perfect for:
  - Simple visualizations
  - Debugging spatial logic
  - Quick data exploration
- Fully scriptable and Python-native

Shapely Geometry Creation

```
from shapely.geometry import Point, LineString, Polygon

p = Point(0, 0)
line = LineString([(0, 0), (1, 1), (1, 0)])
poly = Polygon([(0, 0), (1, 0), (1, 1), (0, 1)])
```

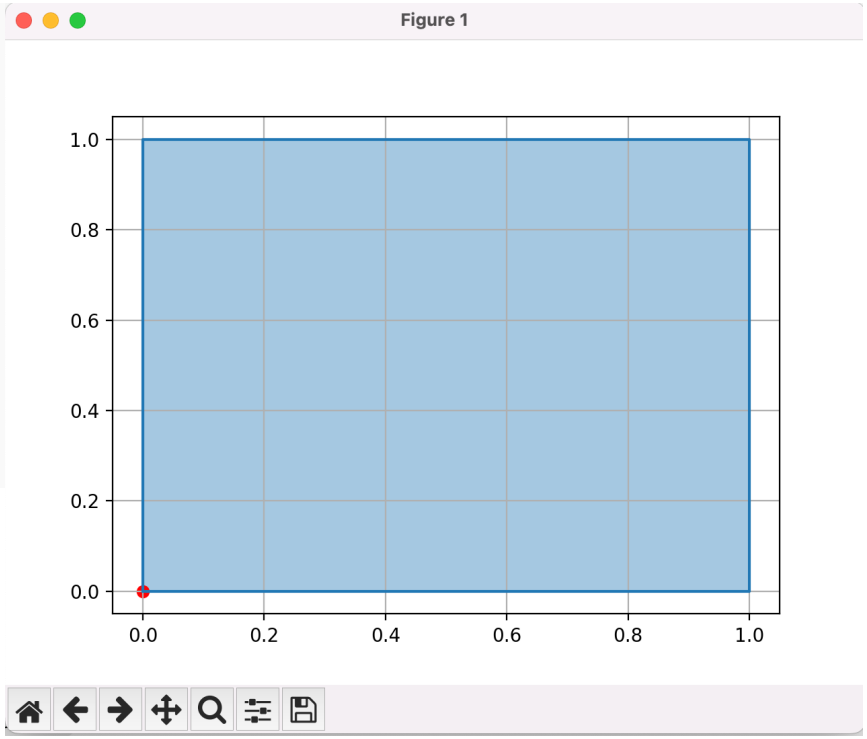
Geometry Analysis

```
print(poly.area)        # 1.0
print(p.within(poly))   # True
print(poly.contains(p)) # True
```

- Other operations:
  - .intersection()
  - .union()
  - .difference()

Plotting with Matplotlib

```
import matplotlib.pyplot as plt

x, y = poly.exterior.xy
plt.plot(x, y)
plt.fill(x, y, alpha=0.4)
plt.scatter([p.x], [p.y], color='red')
plt.grid()
plt.show()
```



Tips for Plotting Geometries

- Point: use scatter
- LineString: use plot
- Polygon: use fill and plot
- Always extract .xy from geometry parts:

```
x, y = geom.xy  # for LineString and Polygon.exterior
```

Handling Multiple Geometries

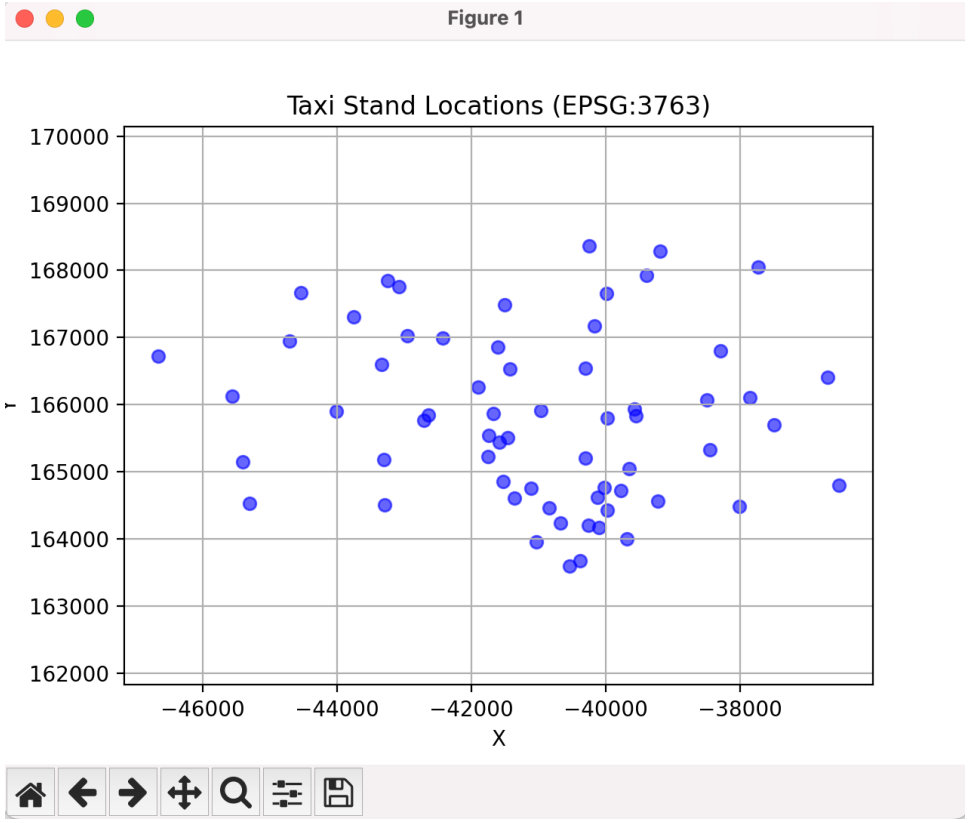
```
points = [Point(x, x**2) for x in range(10)]
xs = [p.x for p in points]
ys = [p.y for p in points]

plt.scatter(xs, ys)
```

- Useful for plotting results from spatial queries

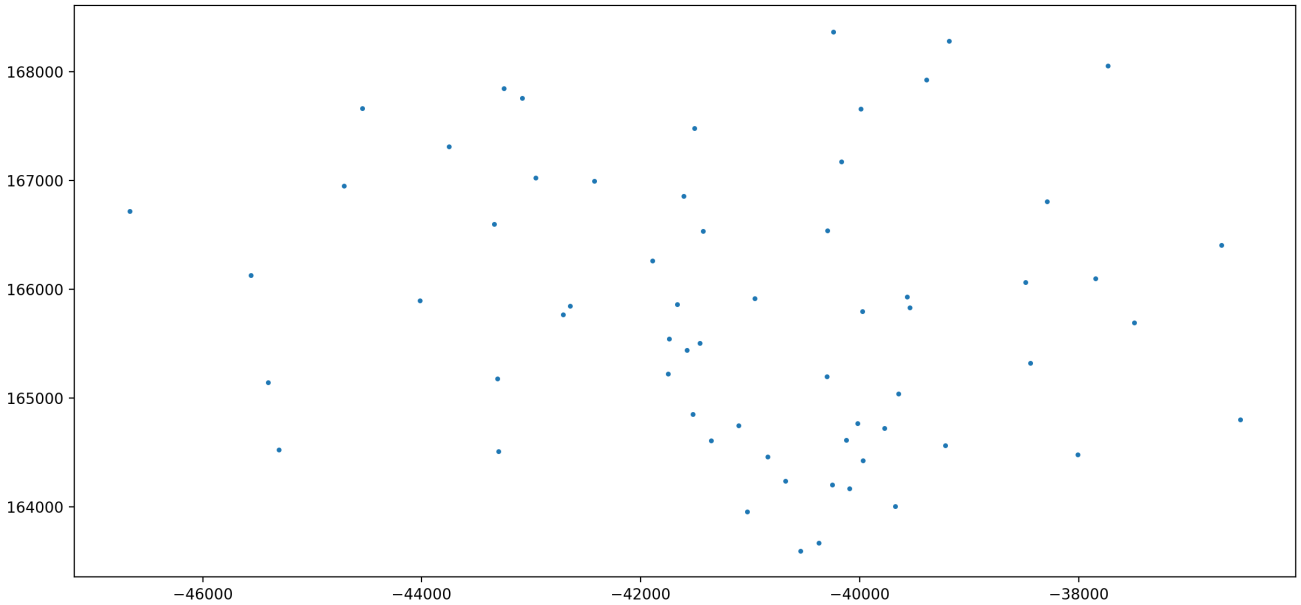
Creating a scatter plot to show the taxi stands of Porto

```
import psycopg2
from shapely import wkb
import matplotlib.pyplot as plt

# Connect and query

conn = psycopg2.connect(dbname="mi", user="michelferreira", password="",
    host="localhost", port="5432")

cur = conn.cursor()
cur.execute("SELECT proj_location FROM taxi_stands")
points = [wkb.loads(row[0]) for row in cur.fetchall()]

# Extract coordinates
xs = [pt.x for pt in points]
ys = [pt.y for pt in points]

# Plot
plt.scatter(xs, ys, color='blue', alpha=0.6)
plt.title("Taxi Stand Locations (EPSG:3763)")
plt.xlabel("X")
plt.ylabel("Y")
plt.grid(True)
plt.axis('equal')
plt.show()
```

Creating a scatter plot to show the taxi stands of Porto



Creating a scatter plot to show the taxi stands of Porto

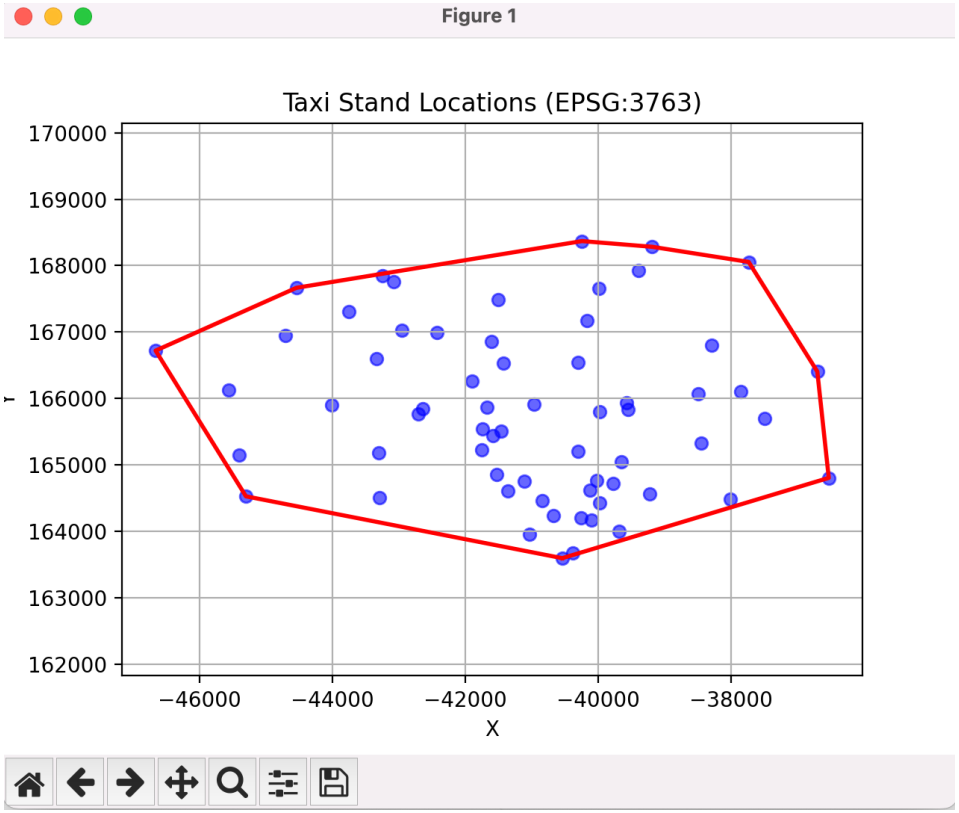


Creating a line plot with the boundary of the convex hull of taxi stands

```
cur.execute("""
    SELECT ST_ConvexHull(ST_Collect(proj_location))
    FROM taxi_stands;
""")
hull_geom = wkb.loads(cur.fetchone()[0])
hx, hy = hull_geom.exterior.xy
```

```
plt.plot(hx, hy, color='red', linewidth=2, label="Convex Hull")
```

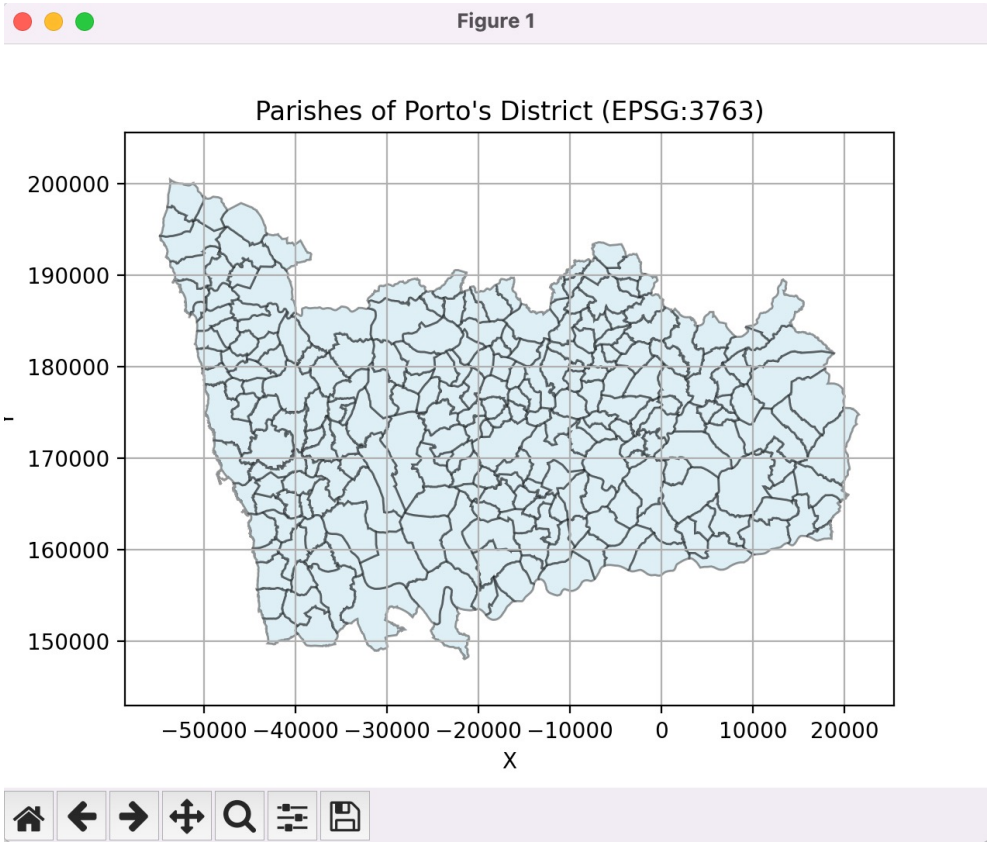
Creating a line plot with the boundary of the convex hull of taxi stands



Visualizing the parishes of the district of Porto

```
import matplotlib.pyplot as plt

# Connect and query

conn = psycopg2.connect(dbname="mi", user="michelferreira", password="",
    host="localhost", port="5432")

cur = conn.cursor()

cur.execute("""SELECT proj_boundary
    FROM cont_aad_caop2018 where distrito = 'PORTO';""")
rows = cur.fetchall()
polygons = [wkb.loads(row[0]) for row in rows]
for poly in polygons:
    x, y = poly.exterior.xy
    plt.fill(x, y, alpha=0.4, edgecolor='black', facecolor='lightblue')

plt.title("Parishes of Porto's District (EPSG:3763)")
plt.xlabel("X")
plt.ylabel("Y")
plt.grid(True)
plt.axis('equal')
plt.show()
```

Visualizing the parishes of the district of Porto



What is contextily?

- A Python module that allows you to:
  - Add **tile-based basemaps** to matplotlib plots
  - Use providers like **OpenStreetMap**, **CartoDB**, **Stamen**, etc.
- Integrates with:
  - matplotlib
  - geopandas
  - shapely

Why use contextily?

- Gives **geographic context** to your plots
- Makes spatial data **easier to interpret**
- Simple API: just one function → add_basemap()
- Downloads tiles from providers (in **EPSG:3857**)
- Plots them as a **background raster** in your matplotlib plot
- Your geometries must also be in **EPSG:3857**

Basic Example

```
import contextily as ctx
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(10, 10))
# Plot your shapely geometries (already reprojected)
ax.fill(x, y, facecolor='lightblue', edgecolor='black')

# Add the basemap
ctx.add_basemap(ax, source=ctx.providers.CartoDB.Positron)

plt.axis('equal')
plt.show()
```

Popular Basemap Providers

- Use ctx.providers.<provider> to select:
- OpenStreetMap.Mapnik
- CartoDB.Positron *(light gray)*
- Stamen.Toner *(black/white, high contrast)*
- Stamen.Terrain
- CartoDB.DarkMatter

```
ctx.add_basemap(ax, source=ctx.providers.OpenStreetMap.Mapnik)
```

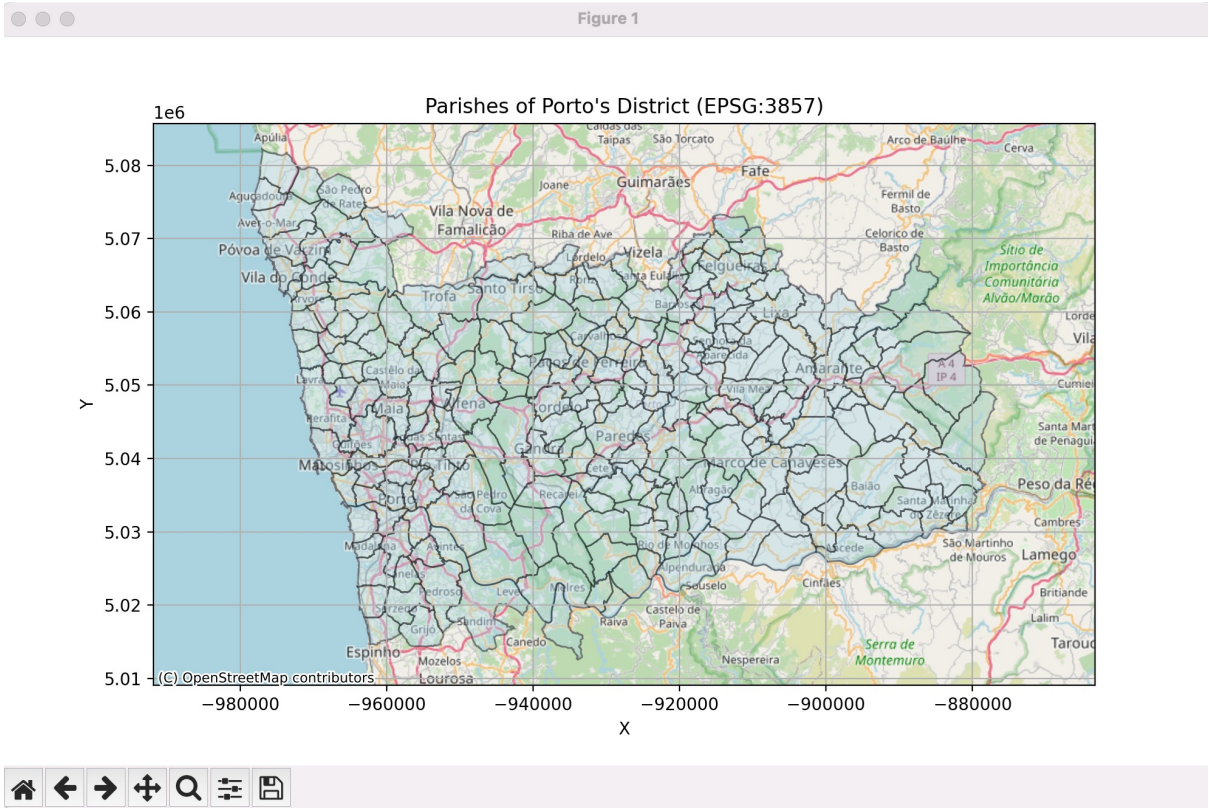
Adjusting Zoom and Extent

```
ctx.add_basemap(ax, zoom=12)
```

- Or fit to a custom extent:

```
ctx.add_basemap(ax, extent=(minx, miny, maxx, maxy))
```

Porto Parishes and Tiles

```
import psycopg2
from shapely import wkb
import matplotlib.pyplot as plt
import contextily as ctx

# Connect and query

conn = psycopg2.connect(dbname="mi", user="michelferreira", password="",
    host="localhost", port="5432")

cur = conn.cursor()

cur.execute("""SELECT st_transform(proj_boundary,3857)
    FROM cont_aad_caop2018 where distrito = 'PORTO';""")
rows = cur.fetchall()
polygons = [wkb.loads(row[0]) for row in rows]

fig, ax = plt.subplots(figsize=(10, 6))

for poly in polygons:
    x, y = poly.exterior.xy
    ax.fill(x, y, alpha=0.4, edgecolor='black', facecolor='lightblue')

plt.title("Parishes of Porto's District (EPSG:3857)")
plt.xlabel("X")
plt.ylabel("Y")
plt.grid(True)
plt.axis('equal')
ctx.add_basemap(ax, crs="EPSG:3857", source=ctx.providers.OpenStreetMap.Mapnik)
plt.show()
```

Porto Parishes and Tiles